

Spec — Telegram Architecture: Primary/Secondary + Bot Lane Cleanup

Created: 2026-06-20 — Claude.ai Status: Ready for `_working/` Priority: High — blocks full EC2 validation

Context

mini-moi now runs on two instances: EC2 (primary, production) and Mac (secondary, test). Telegram currently runs on Mac only via launchd. This creates three problems:

1. If Mac is off, no bot is polling for commands
2. No way to test Telegram flows without sending to production bots
3. Bot lane assignments have known design debt (`!ops` on wrong bot)

This spec resolves all three in one build.

Design Decisions (locked)

One production only. EC2 is primary. Mac is secondary. No data divergence, no duplicate alerts, no command conflicts.

Two bot sets. Production bots on primary (EC2). Test bots on secondary (Mac). Same developer receives both — different bot names make it clear which instance sent what.

Role pointer via environment variable. `MINIMOI_ROLE=primary` or `MINIMOI_ROLE=secondary`. Default is `primary` so EC2 needs no special handling. Mac explicitly set to `secondary`.

Part 1 — Bot Lane Cleanup

Current state (design debt)

- `minimoi_cmd_bot` — handles German drill commands
- `rvsopenbot` — outbound briefings and alerts
- `minimoi_agent_bot` — OpenClaw conversational
- `!ops` command — on wrong bot (tracked debt)

Clean lane assignments

Bot	Direction	Purpose
<code>minimoi_cmd_bot</code>	Inbound	All commands from Robert → system. <code>!ops</code> , German drills, any <code>!</code> prefixed command
<code>rvsopenbot</code>	Outbound	All system → Robert messages. Curator briefings, alerts, notifications, deploy status
<code>minimoi_agent_bot</code>	Bidirectional	OpenClaw conversational agent. Natural language, not <code>!</code> commands

Rule: No bot handles both inbound commands AND outbound alerts. One job per bot. `!ops` moves to `minimoi_cmd_bot`.

Changes required

- Move `!ops` handler from current bot to `minimoi_cmd_bot`
- Audit all `send_message()` calls — confirm they use `rvsopenbot` token, not `cmd_bot` token
- Confirm `minimoi_agent_bot` is isolated to OpenClaw only
- Document the three lanes in `docs/TELEGRAM_ARCHITECTURE.md`

Part 2 — Test Bot Setup

Create three test bots in BotFather

Robert creates these manually (5 minutes in Telegram):

Test bot	Mirrors	Purpose
<code>minimoi_cmd_bot_test</code>	<code>minimoi_cmd_bot</code>	Test inbound commands
<code>rvsopenbot_test</code>	<code>rvsopenbot</code>	Test outbound alerts/briefings
<code>minimoi_agent_bot_test</code>	<code>minimoi_agent_bot</code>	Test OpenClaw conversational

Same chat ID as production — messages go to Robert either way. Bot name in chat header makes it clear which instance sent it.

SSM parameters — two paths

Add to AWS SSM Parameter Store as `SecureString`:

Production path (`/minimoi/production/`) — already exists, update names to match new convention:

<code>/minimoi/production/telegram_cmd_bot_token</code>	← <code>minimoi_cmd_bot</code> token
<code>/minimoi/production/telegram_alert_bot_token</code>	← <code>rvsopenbot</code> token
<code>/minimoi/production/telegram_agent_bot_token</code>	← <code>minimoi_agent_bot</code> token
<code>/minimoi/production/telegram_chat_id</code>	← Robert's chat ID

Test path (`/minimoi/test/`) — new, create these:

<code>/minimoi/test/telegram_cmd_bot_token</code>	← <code>minimoi_cmd_bot_test</code> token
<code>/minimoi/test/telegram_alert_bot_token</code>	← <code>rvsopenbot_test</code> token
<code>/minimoi/test/telegram_agent_bot_token</code>	← <code>minimoi_agent_bot_test</code> token
<code>/minimoi/test/telegram_chat_id</code>	← Robert's chat ID (same)

Part 3 — Primary/Secondary Role System

Environment variable

```
bash

# EC2 .env (primary — production)
MINIMOI_ROLE=primary

# Mac .env (secondary — test)
MINIMOI_ROLE=secondary
```

Default is `primary` — EC2 requires no special handling beyond what's already in `.env`.
Mac explicitly set to `secondary`.

`is_primary()` helper

Add to shared utilities (`utils/role.py` or equivalent):

```
python
```

```

import os

def is_primary() -> bool:
    """
    Returns True if this instance is the primary (production) instance.
    Default is primary – secondary must be explicitly set.
    """
    return os.environ.get('MINIMOI_ROLE', 'primary') == 'primary'

def get_telegram_path() -> str:
    """
    Returns SSM path prefix for Telegram credentials based on role.
    Primary uses production bots. Secondary uses test bots.
    """
    return 'production' if is_primary() else 'test'

```

get_secret() update for path-aware Telegram tokens

python

```

def get_telegram_token(bot_name: str) -> str:
    """
    Reads Telegram bot token from SSM using role-appropriate path.
    bot_name: 'cmd_bot', 'alert_bot', or 'agent_bot'
    """
    path = get_telegram_path()
    return get_secret(f'telegram_{bot_name}_token', ssm_path=f'/minimoi/{path}/')

```

Guards on all outbound actions

Add `is_primary()` check before every outbound Telegram call and every scheduled job:

python

```
# Curator briefing delivery
if is_primary():
    send_telegram_briefing(briefing)
else:
    logger.info("Secondary instance – briefing suppressed, not sending to Telegram")

# Telegram bot startup
if not is_primary():
    # Secondary still runs – but uses test bots
    logger.info("Secondary instance – using test Telegram bots")
    # bot initialization continues with test tokens

# Scheduled scoring loops
if is_primary():
    run_curator_scoring_loop()
else:
    logger.info("Secondary instance – scoring loop suppressed")
```

Important: Secondary does NOT suppress the bot entirely — it runs the bot with test tokens. This gives Robert a real Telegram testing environment on Mac.

What secondary suppresses:

- Scheduled outbound jobs (briefings, alerts)
- Curator scoring loop
- Any cron/periodic tasks

What secondary still runs:

- Web app (portal, German, Curator) — for testing
- Telegram bot — but on test bots only
- Manual commands — fully functional on test bots

Part 4 — EC2 Bot Deployment

Add telegram-bot to docker-compose.prod.yml

```
yaml
```

```
telegram-bot:
  build:
    context: .
    dockerfile: telegram/Dockerfile
  env_file: /opt/minimoi/.env
  environment:
    - MINIMOI_ROLE=primary
  restart: unless-stopped
  depends_on:
    - postgres
  networks:
    - minimoi-network
  logging:
    driver: "json-file"
    options:
      max-size: "10m"
      max-file: "3"
```

telegram/Dockerfile

```
dockerfile

FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY telegram/ .
COPY utils/ ./utils/
CMD ["python", "telegram_bot.py"]
```

Mac launchd update

Remove individual bot launchd entries. The Mac startup script handles bot startup:

```
bash
```

```
#!/bin/bash
# start_minimoi.sh
export MINIMOI_ROLE=secondary
cd ~/Projects/personal-ai-agents
git checkout staging
git pull origin staging
docker-compose -f docker-compose.dev.yml up -d
echo "mini-moi secondary started on $(date)"
```

`docker-compose.dev.yml` includes the telegram-bot service. On Mac it picks up `MINIMOI_ROLE=secondary` and uses test bots.

Part 5 — Documentation

Create `docs/TELEGRAM_ARCHITECTURE.md`:

markdown

Telegram Architecture

Bot Lanes

```
|
  Bot
|
  Direction
|
  Purpose
|
|
-----
|
-----
|
-----
|
|
  minimoi_cmd_bot
|
  Inbound
|
  Commands (!ops, !german, etc.)
|
|
  rvsopenbot
|
  Outbound
|
  Briefings, alerts, notifications
|
|
  minimoi_agent_bot
|
  Bidirectional
|
  OpenClaw conversational
|
```

Primary/Secondary

MINIMOI_ROLE=primary → production bots, all jobs run

MINIMOI_ROLE=secondary → test bots, scheduled jobs suppressed

Test Bots

minimoi_cmd_bot_test, rvsopenbot_test, minimoi_agent_bot_test
Created in BotFather. Same chat ID as production.
Used on Mac (secondary) for full Telegram testing without affecting production bot streams.

SSM Paths

Production: /minimoi/production/telegram_*
Test: /minimoi/test/telegram_*

Commit Sequence

Four commits in order:

1. Bot lane cleanup – move !ops to cmd_bot, audit send_message calls
2. Add is_primary(), get_telegram_path(), role-aware get_telegram_token()
3. Add telegram-bot to docker-compose.prod.yml and docker-compose.dev.yml
4. Add MINIMOI_ROLE to .env files, update Mac startup script, add docs

Definition of Done

- ☐ !ops command handled by minimoi_cmd_bot (not other bots)
- ☐ All outbound alerts use rvsopenbot token only
- ☐ minimoi_agent_bot isolated to OpenClaw only
- ☐ Three test bots created in BotFather by Robert
- ☐ SSM parameters created for /minimoi/test/ path (4 parameters)
- ☐ SSM parameters updated for /minimoi/production/ path (renamed to new convention)
- ☐ is_primary() and get_telegram_path() in shared utils
- ☐ get_telegram_token() reads from role-appropriate SSM path
- ☐ MINIMOI_ROLE=primary in EC2.env
- ☐ MINIMOI_ROLE=secondary in Mac.env
- ☐ telegram-bot container in docker-compose.prod.yml
- ☐ telegram-bot container in docker-compose.dev.yml
- ☐ Mac startup script sets MINIMOI_ROLE=secondary

- ☐ Scheduled jobs guarded with is_primary() check
- ☐ docs/TELEGRAM_ARCHITECTURE.md committed
- ☐ Verified: send command on Mac → test bot responds, production bot silent
- ☐ Verified: Curator briefing on EC2 → production bot sends, Mac silent
- ☐ Verified: Mac bot fully functional with test bots (commands work)
- ☐ Verified: EC2 bot polling after docker-compose up

Commit Message

Telegram: primary/secondary roles, test bot set, 3-bot lane cleanup, EC2 bot deployment. MINIMOI_ROLE controls production vs test bots.

*Spec · 2026-06-20 · Claude.ai Related: docs/PRODUCTION_CUTOVER_PLAN.md Robert
action required: create 3 test bots in BotFather before Claude Code builds*